

The Singular Value Decomposition (SVD)

Lecture Notes

Zamir Hussain
Department of Mathematics
University of Wah

January 13, 2026

Based on Lecture Series by Mahesh Huddar

Abstract

These comprehensive lecture notes cover Singular Value Decomposition (SVD), a fundamental matrix factorization technique with extensive applications in machine learning, dimensionality reduction, image processing, and data science. The notes include step-by-step examples, mathematical derivations, practical applications, and practice problems to facilitate deep understanding.

1 Motivation: Why We Need SVD

In previous lectures, we have explored several matrix factorizations like LU and QR decomposition. You might ask: why do we need another one? The motivation for the Singular Value Decomposition (SVD) arises from the limitations of standard linear algebra concepts when dealing with real-world, noisy data.

While theoretical concepts like “rank” or “invertibility” are binary (a matrix is either one or the other), practical applications demand a more nuanced view. The SVD provides this nuance and is driven by three powerful applications:

1. **Numerical Stability and Sensitivity Analysis:** Real-world matrices are rarely truly singular, but they can be “almost singular.” SVD helps us quantify *how close* a matrix is to being singular, which is crucial for understanding how small errors in input data can lead to huge errors in the output.
2. **Data Compression and Low-Rank Approximation:** Many large data matrices (like user-movie ratings) are “almost low rank.” SVD provides the *optimal* way to factor such matrices into much smaller, manageable pieces, leading to massive gains in storage and computational speed.
3. **Dimensionality Reduction and PCA:** SVD is the mathematical engine behind **Principal Component Analysis (PCA)**, allowing us to find the best-fitting low-dimensional subspaces for complex, high-dimensional data points.

2 The Challenge with “Almost Singular” Matrices

Let’s explore the first motivation more deeply. Consider solving a simple equation $ax = b$. If a is a very small number, say $a = 0.002$, and our measurement of b has a tiny error, the solution x becomes highly unstable.

Example Problem

[Sensitivity to Small Perturbations] Suppose the true equation is $0.002x = 1$, for which the correct solution is $x = 500$. Now, imagine our measurement of a is not exact and has an error of ± 0.001 .

✔ Step-by-Step Solution

1. **Case 1 (Upper Error Bound):** Let $a = 0.002 + 0.001 = 0.003$.

$$0.003x = 1 \implies x = \frac{1}{0.003} \approx 333.3$$

2. **Case 2 (Lower Error Bound):** Let $a = 0.002 - 0.001 = 0.001$.

$$0.001x = 1 \implies x = \frac{1}{0.001} = 1000$$

💡 Key Idea

A tiny uncertainty in the coefficient a (from 0.001 to 0.003) leads to a massive uncertainty in the solution x , which ranges from 333 to 1000!

In higher dimensions, this problem is less obvious but more dangerous. A matrix A in $A\mathbf{x} = \mathbf{b}$ might have seemingly normal numbers, but if its columns are nearly linearly dependent, it is “almost singular.” SVD gives us a way to measure this instability through its singular values.

3 Defining the Singular Value Decomposition (SVD)

Theorem

[Singular Value Decomposition] Any $m \times n$ matrix A with $\text{rank}(A) = r$ can be factored into:

$$A = U\Sigma V^T$$

where:

- U is an $m \times m$ **orthogonal matrix** ($U^T U = I_m$). Its columns, $\mathbf{u}_i$, are the **left singular vectors**. The first r columns, $\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$, form an orthonormal basis for the column space of A .
- Σ is an $m \times n$ rectangular **diagonal matrix**. Its diagonal entries, σ_i , are the **singular values** of A . They are non-negative and ordered by convention: $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$, with $\sigma_{r+1} = \dots = 0$.
- V is an $n \times n$ **orthogonal matrix** ($V^T V = I_n$). Its columns, $\mathbf{v}_i$, are the **right singular vectors**. The first r columns, $\{\mathbf{v}_1, \dots, \mathbf{v}_r\}$, form an orthonormal basis for the row space of A .

SVD Matrix Dimensions

Matrix	Symbol	Dimension
Original Matrix	A	$m \times n$
Left Singular Vectors	U	$m \times m$
Singular Values	Σ	$m \times n$
Right Singular Vectors (transposed)	V^T	$n \times n$

Key Idea

[Geometric Interpretation] The SVD tells us that any linear transformation A can be broken down into three simple operations:

1. A **rotation** or reflection in the domain space (given by V^T).
2. A **scaling** along the new coordinate axes (given by Σ).
3. A **rotation** or reflection in the co-domain space (given by U).

The fundamental relationship connecting these components is:

$$A\mathbf{v}_i = \sigma_i \mathbf{u}_i \quad \text{for } i = 1, \dots, r$$

This means that A maps the i -th right singular vector to the i -th left singular vector, scaled by the i -th singular value.

4 How to Compute the SVD

The SVD is deeply connected to the eigenvalues and eigenvectors of two related symmetric matrices: $A^T A$ and AA^T .

4.1 Procedure for finding U, Σ, V

1. Compute V and Σ from $A^T A$:

- Calculate the symmetric $n \times n$ matrix $B = A^T A$.
- Find the eigenvalues λ_i and corresponding orthonormal eigenvectors of B .
- The **right singular vectors** ($\mathbf{v}_i$) are these eigenvectors. Form the matrix $V = [\mathbf{v}_1 | \dots | \mathbf{v}_n]$.
- The **singular values** (σ_i) are the square roots of the eigenvalues: $\sigma_i = \sqrt{\lambda_i}$. Form the diagonal matrix Σ with these values, ordered from largest to smallest.

2. Compute U :

- Use the formula $A\mathbf{v}_i = \sigma_i \mathbf{u}_i$ to find the first r **left singular vectors** ($\mathbf{u}_i$):

$$\mathbf{u}_i = \frac{1}{\sigma_i} A\mathbf{v}_i \quad (\text{for all } \sigma_i > 0)$$

- If more columns are needed for U (i.e., if $m > r$), find the remaining columns by extending $\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$ to a full orthonormal basis for $\mathbb{R}^m$. This can be done by finding a basis for the null space of A^T and using the Gram-Schmidt process.

4.2 Step-by-Step Calculation Example

Example Problem

Find the Singular Value Decomposition of the matrix $A = \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix}$.

✓ Step-by-Step Solution

Step 1: Compute $A^T A$

$$A^T A = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

Step 2: Find Eigenvalues and Eigenvectors of $A^T A$ to get Σ and V

- Eigenvalues:** The characteristic equation is $\det(A^T A - \lambda I) = 0$.

$$(2 - \lambda)^2 - 1^2 = 0 \implies (2 - \lambda) = \pm 1$$

The eigenvalues are $\lambda_1 = 3$ and $\lambda_2 = 1$.

- Singular Values (Σ):**

$$\sigma_1 = \sqrt{3}, \quad \sigma_2 = \sqrt{1} = 1$$

Since A is 3×2 , Σ is also a 3×2 matrix:

$$\Sigma = \begin{pmatrix} \sqrt{3} & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}$$

• **Eigenvectors (V):**

– For $\lambda_1 = 3$: $(A^T A - 3I)\mathbf{v}_1 = \begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \mathbf{0}$. This gives $x = y$. The normalized eigenvector is $\mathbf{v}_1 = \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix}$.

– For $\lambda_2 = 1$: $(A^T A - 1I)\mathbf{v}_2 = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \mathbf{0}$. This gives $x = -y$. The normalized eigenvector is $\mathbf{v}_2 = \begin{pmatrix} 1/\sqrt{2} \\ -1/\sqrt{2} \end{pmatrix}$.

$$V = \begin{pmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & -1/\sqrt{2} \end{pmatrix}$$

Step 3: Compute the Left Singular Vectors (U)

• For $\sigma_1 = \sqrt{3}$:

$$\mathbf{u}_1 = \frac{1}{\sigma_1} A \mathbf{v}_1 = \frac{1}{\sqrt{3}} \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix} = \frac{1}{\sqrt{6}} \begin{pmatrix} 2 \\ 1 \\ 1 \end{pmatrix}$$

• For $\sigma_2 = 1$:

$$\mathbf{u}_2 = \frac{1}{\sigma_2} A \mathbf{v}_2 = \frac{1}{1} \begin{pmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} \\ -1/\sqrt{2} \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ -1 \\ 1 \end{pmatrix}$$

• We need a third vector $\mathbf{u}_3$ for the 3×3 matrix U that is orthogonal to both $\mathbf{u}_1$ and $\mathbf{u}_2$. We can find it by finding a basis vector for the null space of A^T . Solving $A^T \mathbf{x} = 0$:

$$\begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \implies x + z = 0, x + y = 0 \implies \mathbf{x} = \begin{pmatrix} 1 \\ -1 \\ -1 \end{pmatrix}$$

Normalizing this gives $\mathbf{u}_3 = \frac{1}{\sqrt{3}} \begin{pmatrix} 1 \\ -1 \\ -1 \end{pmatrix}$.

Finally, we assemble the full U matrix:

$$U = \begin{pmatrix} 2/\sqrt{6} & 0 & 1/\sqrt{3} \\ 1/\sqrt{6} & -1/\sqrt{2} & -1/\sqrt{3} \\ 1/\sqrt{6} & 1/\sqrt{2} & -1/\sqrt{3} \end{pmatrix}$$

The complete SVD is $A = U \Sigma V^T$ with the matrices found above.

5 Example 1: 2×2 Square Matrix

Example Problem

Given: $A = \begin{bmatrix} 1 & 7 \\ 1 & 7 \end{bmatrix}$

Task: Find SVD decomposition $A = U\Sigma V^T$

✓ Step-by-Step Solution

5.1 Step 1: Compute $A^T A$ and AA^T

$$\begin{aligned} A^T &= \begin{bmatrix} 1 & 1 \\ 7 & 7 \end{bmatrix} \\ AA^T &= \begin{bmatrix} 1 & 7 \\ 1 & 7 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 7 & 7 \end{bmatrix} = \begin{bmatrix} 50 & 50 \\ 50 & 50 \end{bmatrix} \\ A^T A &= \begin{bmatrix} 1 & 1 \\ 7 & 7 \end{bmatrix} \begin{bmatrix} 1 & 7 \\ 1 & 7 \end{bmatrix} = \begin{bmatrix} 2 & 14 \\ 14 & 98 \end{bmatrix} \end{aligned}$$

5.2 Step 2: Find Eigenvalues of $A^T A$

$$\begin{aligned} \det(A^T A - \lambda I) &= \det \begin{bmatrix} 2 - \lambda & 14 \\ 14 & 98 - \lambda \end{bmatrix} = 0 \\ (2 - \lambda)(98 - \lambda) - 196 &= 0 \\ \lambda^2 - 100\lambda &= 0 \\ \lambda(\lambda - 100) &= 0 \\ \lambda_1 = 100, \quad \lambda_2 = 0 \end{aligned}$$

5.3 Step 3: Find Eigenvectors of $A^T A$

For $\lambda_1 = 100$:

$$\begin{aligned} (A^T A - 100I)\mathbf{v}_1 &= 0 \\ \begin{bmatrix} -98 & 14 \\ 14 & -2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} &= \begin{bmatrix} 0 \\ 0 \end{bmatrix} \\ 14x - 2y &= 0 \Rightarrow y = 7x \end{aligned}$$

Let $x = 1 \Rightarrow y = 7$, so $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 7 \end{bmatrix}$

For $\lambda_2 = 0$:

$$\begin{aligned} (A^T A - 0I)\mathbf{v}_2 &= 0 \\ \begin{bmatrix} 2 & 14 \\ 14 & 98 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} &= \begin{bmatrix} 0 \\ 0 \end{bmatrix} \\ 2x + 14y &= 0 \Rightarrow x = -7y \end{aligned}$$

Let $y = 1 \Rightarrow x = -7$, so $\mathbf{v}_2 = \begin{bmatrix} -7 \\ 1 \end{bmatrix}$

5.4 Step 4: Normalize Eigenvectors

$$\begin{aligned}\|\mathbf{v}_1\| &= \sqrt{1^2 + 7^2} = \sqrt{50} = 7.071 \\ \mathbf{v}_1^{\text{norm}} &= \begin{bmatrix} 1/7.071 \\ 7/7.071 \end{bmatrix} = \begin{bmatrix} 0.141 \\ 0.990 \end{bmatrix} \\ \|\mathbf{v}_2\| &= \sqrt{(-7)^2 + 1^2} = \sqrt{50} = 7.071 \\ \mathbf{v}_2^{\text{norm}} &= \begin{bmatrix} -7/7.071 \\ 1/7.071 \end{bmatrix} = \begin{bmatrix} -0.990 \\ 0.141 \end{bmatrix}\end{aligned}$$

5.5 Step 5: Construct V and V^T

$$V = \begin{bmatrix} 0.141 & -0.990 \\ 0.990 & 0.141 \end{bmatrix}, \quad V^T = \begin{bmatrix} 0.141 & 0.990 \\ -0.990 & 0.141 \end{bmatrix}$$

5.6 Step 6: Construct Σ

$$\begin{aligned}\sigma_1 &= \sqrt{100} = 10, \quad \sigma_2 = \sqrt{0} = 0 \\ \Sigma &= \begin{bmatrix} 10 & 0 \\ 0 & 0 \end{bmatrix}\end{aligned}$$

5.7 Step 7: Construct U

Using $u_i = \frac{1}{\sigma_i} A \mathbf{v}_i$:

For $i = 1$:

$$u_1 = \frac{1}{10} \begin{bmatrix} 1 & 7 \\ 1 & 7 \end{bmatrix} \begin{bmatrix} 0.141 \\ 0.990 \end{bmatrix} = \frac{1}{10} \begin{bmatrix} 7.14 \\ 7.14 \end{bmatrix} = \begin{bmatrix} 0.714 \\ 0.714 \end{bmatrix}$$

For $i = 2$: Since $\sigma_2 = 0$, find u_2 orthogonal to u_1 :

$$\begin{aligned}u_2 &= \begin{bmatrix} -0.714 \\ 0.714 \end{bmatrix} \\ U &= \begin{bmatrix} 0.714 & -0.714 \\ 0.714 & 0.714 \end{bmatrix}\end{aligned}$$

5.8 Step 8: Verification

$$U \Sigma V^T = \begin{bmatrix} 0.714 & -0.714 \\ 0.714 & 0.714 \end{bmatrix} \begin{bmatrix} 10 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} 0.141 & 0.990 \\ -0.990 & 0.141 \end{bmatrix} = \begin{bmatrix} 1 & 7 \\ 1 & 7 \end{bmatrix} = A$$

6 Example 2: 2×3 Rectangular Matrix

Example Problem

Given: $A = \begin{bmatrix} 1 & -2 & 3 \\ 3 & 2 & -1 \end{bmatrix}$

Task: Find SVD decomposition

✓ Step-by-Step Solution

6.1 Step 1: Compute $A^T A$

$$A^T A = \begin{bmatrix} 10 & 4 & 0 \\ 4 & 8 & -8 \\ 0 & -8 & 10 \end{bmatrix}$$

6.2 Step 2: Find Eigenvalues

$$\det(A^T A - \lambda I) = \lambda^3 - 28\lambda^2 + 180\lambda = 0$$

$$\lambda(\lambda - 10)(\lambda - 18) = 0$$

$$\lambda_1 = 18, \quad \lambda_2 = 10, \quad \lambda_3 = 0$$

6.3 Step 3: Find Eigenvectors

- For $\lambda_1 = 18$: $\mathbf{v}_1 = \begin{bmatrix} -1/3 \\ -2/3 \\ 2/3 \end{bmatrix}$
- For $\lambda_2 = 10$: $\mathbf{v}_2 = \begin{bmatrix} 2/\sqrt{5} \\ 0 \\ 1/\sqrt{5} \end{bmatrix}$
- For $\lambda_3 = 0$: $\mathbf{v}_3 = \begin{bmatrix} -2/(3\sqrt{5}) \\ \sqrt{5}/3 \\ 4/(3\sqrt{5}) \end{bmatrix}$

6.4 Step 4: Construct V and V^T

$$V = \begin{bmatrix} -1/3 & 2/\sqrt{5} & -2/(3\sqrt{5}) \\ -2/3 & 0 & \sqrt{5}/3 \\ 2/3 & 1/\sqrt{5} & 4/(3\sqrt{5}) \end{bmatrix}$$

6.5 Step 5: Construct Σ

$$\Sigma = \begin{bmatrix} \sqrt{18} & 0 & 0 \\ 0 & \sqrt{10} & 0 \end{bmatrix}$$

6.6 Step 6: Construct U

$$U = \begin{bmatrix} 1/\sqrt{2} & 1/\sqrt{2} \\ -1/\sqrt{2} & 1/\sqrt{2} \end{bmatrix}$$

6.7 Step 7: Verification

$$U\Sigma V^T = A$$

7 Application: Low-Rank Approximation and Data Compression

This is where the SVD shines for large datasets. The **Eckart-Young Theorem** states that the best rank- k approximation to a matrix A (in the sense of minimizing error) is given by the

truncated SVD.

Definition

[Truncated SVD] To get a rank- k approximation, we keep only the first k singular values and the corresponding first k columns of U and V .

$$A_k = U_k \Sigma_k V_k^T$$

where U_k is an $m \times k$ matrix, Σ_k is a $k \times k$ diagonal matrix, and V_k^T is a $k \times n$ matrix. This is the optimal $A \approx BC$ factorization.

Key Idea

[Netflix Example] Imagine a matrix A of user ratings: 10 million users ($m = 10^7$) by 10,000 movies ($n = 10^4$).

- **Full Matrix Storage:** $m \times n = 10^{11}$ entries. At 4 bytes per entry, this is **400 GB**.
- **Low-Rank Approximation (assume rank $k = 10$):**
 - We store U_{10} ($10^7 \times 10$), Σ_{10} (10×10), and V_{10}^T (10×10^4).
 - **Total entries:** $(10^7 \times 10) + (10 \times 10) + (10 \times 10^4) \approx 10^8$.
 - **SVD Storage:** 10^8 entries at 4 bytes each is **0.4 GB**.

This represents a **1000x reduction in storage** and a similar reduction in computation time for operations like matrix-vector products, making analysis feasible.

Key Idea

[Connection to PCA] The SVD is the mathematical engine behind Principal Component Analysis (PCA). When we have a set of high-dimensional data points, PCA aims to find a lower-dimensional subspace that best fits the data.

- The right singular vectors ($\mathbf{v}_i$ in V) are the **principal components**, representing the directions of maximum variance in the data.
- The left singular vectors (U) combined with the singular values (Σ) give the coordinates of the data points in this new, reduced-dimensional basis.

Finding the top k principal components is equivalent to finding the best rank- k SVD approximation.

8 Applications in Machine Learning & Data Science

8.1 Dimensionality Reduction (PCA)

- SVD is the computational backbone of Principal Component Analysis (PCA)
- Retain top- k singular values for approximation: $A_k = U_k \Sigma_k V_k^T$
- Reduces noise and computational complexity
- Preserves maximum variance with minimum dimensions

8.2 Image Compression

- Represent image as pixel matrix
- Apply SVD, keep only significant singular values
- Storage savings: Store U_k, Σ_k, V_k^T instead of full matrix
- Compression ratio: $\frac{mk+k+nk}{mn}$ for rank- k approximation

8.3 Recommendation Systems

- User-item interaction matrix (sparse)
- SVD decomposes into user and item latent features
- Predict missing ratings: $\hat{r}_{ui} = u_u^T \Sigma v_i$
- Used in Netflix Prize competition

8.4 Natural Language Processing

- Latent Semantic Analysis (LSA)
- Term-document matrix decomposition
- Reveals semantic relationships between words
- Word similarity in reduced vector space

8.5 Noise Reduction

- Small singular values often correspond to noise
- Zero out small σ_i to remove noise
- Applications: Signal processing, finance, bioinformatics

9 Practice Problems

Example Problem

[Problem 1: Basic SVD] Given $A = \begin{bmatrix} 3 & 0 \\ 0 & -2 \end{bmatrix}$, find its SVD decomposition.

Example Problem

[Problem 2: Rectangular Matrix] Find SVD of $A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$.

Example Problem

[Problem 3: Application - Image Compression] A 500×500 image has SVD with singular values: $[200, 150, 100, 50, 10, 5, 1, 0.5, \dots]$

- (a) How much storage is needed for rank-5 approximation?
- (b) What percentage of "energy" (sum of squared singular values) is preserved?

Example Problem

[Problem 4: Conceptual Understanding] Explain why SVD of a symmetric matrix gives $U = V$ (up to sign changes).

Example Problem

[Problem 5: Computational Efficiency] Given $A = \begin{bmatrix} 2 & 2 \\ -1 & 1 \end{bmatrix}$:

- (a) Find its SVD
- (b) Use SVD to compute A^{10} efficiently

10 Solutions and Hints

10.1 Problem 1 Hint

For diagonal matrix $A = \begin{bmatrix} 3 & 0 \\ 0 & -2 \end{bmatrix}$:

- Singular values are absolute values of diagonal entries
- U and V contain identity or sign adjustments

10.2 Problem 2 Solution Steps

1. Compute $A^T A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$
2. Eigenvalues: $\lambda_1 = 3, \lambda_2 = 1$
3. Proceed with standard SVD procedure

10.3 Problem 3 Solution

- (a) Storage for rank-5: $500 \times 5 + 5 + 500 \times 5 = 5005$ values vs 250000 original
- (b) Energy preserved: $\frac{200^2 + 150^2 + 100^2 + 50^2 + 10^2}{\sum \sigma_i^2} \times 100\%$

10.4 Problem 4 Explanation

For symmetric A , we have $A = A^T$. Then $A^T A = A^2$, so eigenvectors of $A^T A$ are same as eigenvectors of A , making $U = V$.

10.5 Problem 5 Hint

- (a) Compute SVD normally
- (b) $A^{10} = U \Sigma^{10} V^T$ (since U and V are orthogonal)

11 Summary

💡 Key Idea		
	Concept	Key Insight
[Key Takeaways]	SVD Formula	$A = U\Sigma V^T$: Rotation-Scaling-Rotation
	Singular Values	$\sigma_i = \sqrt{\lambda_i(A^T A)}$, always non-negative
	Orthogonality	$U^T U = I, V^T V = I$ (numerical stability)
	Rank Revealing	# non-zero σ_i = rank of A
	Low-Rank Approximation	$A_k = U_k \Sigma_k V_k^T$ (Eckart-Young Theorem)
	Dimensionality Reduction	Keep top- k singular values for PCA
	Image Compression	Store U_k, Σ_k, V_k^T instead of full matrix
	Noise Reduction	Zero small σ_i corresponding to noise
	Recommendation Systems	User-item matrix factorization
	Computational Note	Use $A^T A$ for V , AA^T for U (smaller dimension)

- The SVD ($A = U\Sigma V^T$) is a powerful factorization that provides a nuanced view of a matrix's properties, going beyond binary concepts like rank.
- The **singular values** (σ_i) in Σ are the heart of the SVD. Small singular values indicate the matrix is close to being rank-deficient or numerically unstable.
- The ratio σ_1/σ_r (the **condition number**) is a direct measure of a matrix's sensitivity to numerical errors.
- SVD provides the **best low-rank approximation** to a matrix, making it the premier tool for data compression, noise reduction, and dimensionality reduction (PCA).

A Appendix: Python Implementation

A.1 Basic SVD Computation

```
import numpy as np

def manual_svd(A):
    # Compute A^T A and its eigenvectors
    ATA = np.dot(A.T, A)
    eigvals, eigvecs = np.linalg.eig(ATA)

    # Sort by eigenvalues (descending)
    idx = eigvals.argsort()[::-1]
    eigvals = eigvals[idx]
    V = eigvecs[:, idx]

    # Singular values
    sigma = np.sqrt(eigvals)
```

```

# Construct Sigma matrix
m, n = A.shape
Sigma = np.zeros((m, n))
min_dim = min(m, n)
Sigma[:min_dim, :min_dim] = np.diag(sigma[:min_dim])

# Compute U
U = np.zeros((m, m))
for i in range(min_dim):
    if sigma[i] > 1e-10:
        U[:, i] = np.dot(A, V[:, i]) / sigma[i]

return U, Sigma, V.T

# Compare with NumPy's implementation
A = np.array([[1, 2], [2, 1]])
U_np, S_np, VT_np = np.linalg.svd(A, full_matrices=True)

```

A.2 Low-Rank Approximation

```

def low_rank_approximation(A, k):
    """Return rank-k approximation of A using SVD"""
    U, S, VT = np.linalg.svd(A, full_matrices=False)
    A_k = np.dot(U[:, :k], np.dot(np.diag(S[:k]), VT[:k, :]))
    return A_k

def compression_ratio(A, k):
    """Calculate compression ratio for rank-k approximation"""
    m, n = A.shape
    original_storage = m * n
    compressed_storage = m*k + k + n*k
    return compressed_storage / original_storage

```